

Introduction to Deep Learning

Session 7: Generalization & Regularization

May 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Generalization
2. Generalization Paradox
3. Regularisation Techniques
4. Training Stability
5. Summary

Where We Are

So far we have:

- **Network** $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ - parameterized function
- **Loss** $\mathcal{L}(\theta; \mathcal{D})$ - measures fit on training data
- **Optimization** - algorithms to minimize $\mathcal{L}(\theta; \mathcal{D})$

Open question:

Does minimizing training loss give us a **good model**?

This lecture: what good actually means, and how to get there

What we optimize:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$$

What we actually want: low loss on **new, unseen examples from the population**

$$\mathcal{L}_{\text{pop}}(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} [\ell(f_{\theta}(\mathbf{x}), y)]$$

Generalization gap:

$$\underbrace{\mathcal{L}_{\text{pop}}(\theta)}_{\text{true goal}} - \underbrace{\mathcal{L}_{\text{train}}(\theta)}_{\text{what we minimize}} \geq 0$$

Test set: finite sample we create to *estimate* $\mathcal{L}_{\text{pop}}$ - **measurement tool, not the goal!**

Train / Validation / Test Split

Training set

Fit model parameters

$$\min_{\theta} \mathcal{L}(\theta; \mathcal{D}_{\text{train}})$$

Validation set

Select model, tune
hyperparameters

$$\mathcal{L}(\theta; \mathcal{D}_{\text{val}})$$

Touched many times during
development

Test set

Final honest evaluation

$$\mathcal{L}(\theta; \mathcal{D}_{\text{test}})$$

Touched once, at the end

Common mistake: tuning on test set - reported performance is optimistic

Typical splits: 80/10/10 or 70/15/15 - depends on dataset size

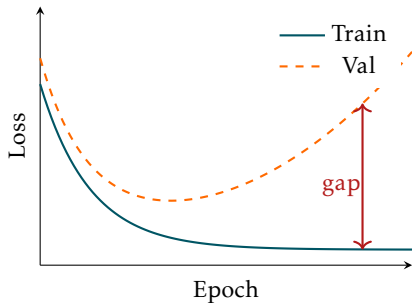
Overfitting and Underfitting

Underfitting: model too simple

- High train loss, high test loss
- Small generalisation gap

Overfitting: model memorises training data

- Low train loss, high test loss
- **Large generalisation gap**



Signal: monitor generalisation gap - **train loss not enough for generalisation**

What To Do

Underfitting (high train loss, small gap)

- Increase model capacity
- Train longer
- Tune learning rate and optimiser
- Reduce regularisation if applied

Overfitting (low train loss, large gap)

- More data - always most effective
- **Regularisation** - this lecture
- Reduce model capacity

Classical view: test error decomposes into two competing terms

Bias - error from wrong assumptions

- Model too simple to capture the data
- High train loss, high test loss
- Underfitting

Variance - sensitivity to training set

- Model fits noise in training data
- Low train loss, high test loss
- Overfitting

Classical prescription: choose model complexity to balance the two

More parameters $\Rightarrow$ lower bias, higher variance $\Rightarrow$ **overfitting**

Classical prediction: more parameters than training examples $\Rightarrow$ **catastrophic overfitting**

Reality in deep learning:

- Deep NNs: huge numbers of params - can memorise training data
- Deep NNs: routinely drive train loss close to zero
- Classical theory: model that nearly memorises train data *cannot generalise*
- Deep NNs: *generalise well in practice* ... hmm ...

Why do deep NNs generalise despite being massively overparameterised?

Generalization Paradox

Many Good Minima

Classical concern: gradient descent finds a local minimum - but which one?

Empirical observation in deep networks:

- Overparameterised networks - vast number of local minima
- Most local minima have very similar loss values
- Different intialisation -> different minima -> *yet generalise similarly*

Why: NNs far more params than needed to fit the data.

Many equivalent solutions exist - which one we find not so critical

Flat vs Sharp Minima

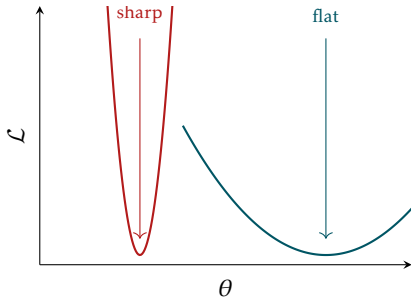
Not all minima generalise equally well

Sharp minimum:

- Loss changes rapidly
- Small distribution shift $\Rightarrow$ loss jumps
- Poor generalisation

Flat minimum:

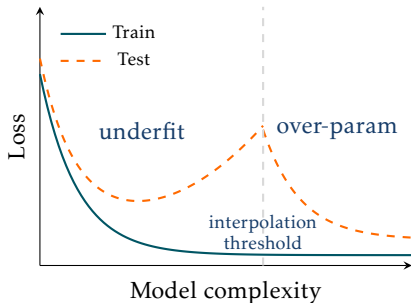
- Loss changes slowly
- Small distribution shift $\Rightarrow$ loss stays low
- Good generalisation



SGD: mini-batching stochastic noise - favours flat minima $\rightarrow$ **implicit regularisation**

Classical bias-variance curve: test error has one optimal complexity - U-shaped curve

Double descent: beyond the interpolation threshold, test error decreases again



- **Underfitting:** too little capacity
- **Interpolation threshold:** maximally rigid - worst generalisation
- **Overparameterised:** smoother interpolation - generalisation recovers

Deep networks operate in overparameterised regime

Regularisation Techniques

Explicit regularisation: add penalty term to the loss

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2} \|\theta\|^2$$

Large weights $\Rightarrow$ model relies heavily on specific features $\Rightarrow$ high variance

L2 regularisation (weight decay)

- Penalises $\|\theta\|^2$ - shrinks all weights
- Smooth, differentiable
- `weight_decay` in `torch.optim`

L1 regularisation

- Penalises $\|\theta\|_1$ - promotes sparsity
- Some weights go exactly to zero
- Useful for feature selection

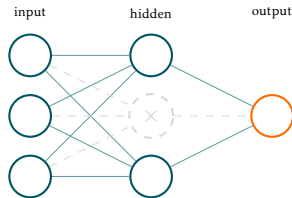
Hyperparameter: $\lambda > 0$ controls strength - tuned on validation set

Deep NNs: baseline, combined with other techniques - less central than in classical ML

Dropout - Training

During training: randomly zero each activation with probability p

- Each forward pass uses a different subnetwork
- Neurons cannot rely on specific others always being present
- Prevents co-adaptation of neurons



Typical values: $p = 0.1-0.5$ - tuned on validation set

During evaluation: all neurons active - outputs scaled by $(1 - p)$ to compensate

`model.train()` vs `model.eval()`:

- `nn.Dropout` - zeroing active in train mode, disabled in eval mode
- Most layers (`nn.Linear`, `nn.ReLU`, ...) - unaffected
- Some other layers also have dual behaviour - covered later

Correct pattern:

```
model.train()
for X_batch, y_batch in train_loader:
    ... # training step
model.eval()
with torch.no_grad():
    ... # validation
```

Forgetting `model.eval()`
is a silent bug - no error,
wrong predictions

Why does dropout work?

With n neurons, dropout implicitly trains 2^n different subnetworks - one per possible dropout mask

- Each mini-batch trains different subnetwork
- Subnetworks share weights - efficient training
- At test time: full network approximates **averaging** over all subnetworks

Ensembles generalise better than single models - averaging reduces variance

Dropout achieves ensemble-like generalisation at the cost of a single model

Idea: apply label-preserving transformations to training examples to create new ones

Key principle - invariance: the transformation must not change the label

- Rotate image of cat $\Rightarrow$ still cat
- Add small noise to patient measurements $\Rightarrow$ same diagnosis
- Model should be invariant to these transformations - augmentation enforces it

Examples:

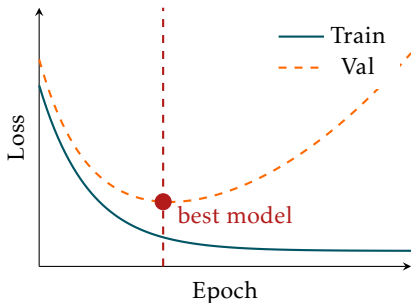
- **Images:** random flips, rotations, crops, colour jitter
- **Tabular data:** Gaussian noise on features, random feature masking

Domain knowledge required - only you know which transformations preserve the label

Early Stopping

Training longer always reduces train loss - point beyond which generalisation gap grows

- Monitor validation loss each epoch
- Save model weights at best validation loss
- Stop when validation loss has not improved for k epochs - **patience**
- Deploy the saved model, not the final one



No change to model or loss function - all ingredients already in the training loop

Training Stability

Regularisation: can use large, overparameterised networks without overfitting :)

New problem: deep NNs training instabilities - more layers amplify small issues :(

- **Internal covariate shift** - activation distributions shift as parameters update
- **Vanishing/exploding grads** - signals decay or amplify exponentially through layers
- **Dying ReLU** - neurons get stuck outputting zero permanently

Not overfitting problems - regularisation does not help here

Problem: as weights update, the input distribution to each layer changes continuously

- Layer l learns to process inputs with a certain distribution
- Layer $l - 1$ updates its weights - distribution shifts
- Layer l must constantly re-adapt - a moving target
- Effect amplifies with depth - later layers more instability

Consequences:

- Slower convergence
- Need for lower learning rates
- High sensitivity to initialisation

Solution: explicitly normalise activations at each layer - **Batch Normalisation**

Idea: normalise activations within each mini-batch -> distributions stable across layers

$$\hat{h}_j = \frac{h_j - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad \tilde{h}_j = \gamma \hat{h}_j + \beta$$

μ_B, σ_B^2 - mini-batch mean and variance γ, β - learnable scale and shift

`model.train():`

- Normalise using mini-batch stats
- Accumulate running mean and var

`model.eval():`

- Normalise using running stats
- Deterministic - no batch dependence

Benefits: stable training, allows higher learning rates, less sensitive to initialisation

Side effect: mini-batch noise introduces mild regularisation - but not main motivation

Vanishing and Exploding Gradients

Backprop multiplies Jacobians backwards - effect compounds, earliest layers affected most

$$\frac{\partial \mathcal{L}}{\partial \theta^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \cdot \frac{\partial \mathbf{h}^{(L)}}{\partial \mathbf{h}^{(L-1)}} \cdots \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \theta^{(1)}}$$

Vanishing (factors < 1)

- Gradient $\rightarrow 0$ at earliest layers
- Early layers stop learning
- Deep NNs effectively shallow

Exploding (factors > 1)

- Gradient $\rightarrow \infty$ at earliest layers
- Single large step destroys weights
- Training diverges

Solutions: Xavier/He init (Session 5); batch norm; gradient clipping (RNN, Transformers)

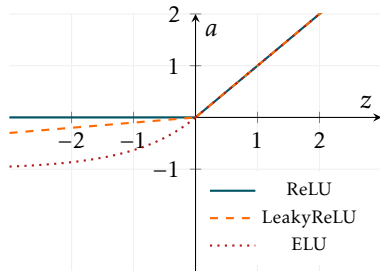
Alarm: ReLU has zero gradient for all negative pre-activations - **half the input space**

$$\frac{\partial \text{ReLU}(z)}{\partial z} = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Relief: neuron dies if pre-activation negative for **all** training examples **simultaneously**

- Pre-activations vary across examples - some positive, some negative
- Average gradient is nonzero - diversity in data saves us
- **ReLU remains solid baseline**

Active research area: many alternatives handle the negative region differently



- **LeakyReLU** - small negative slope α
- **ELU** - smooth exponential region
- **GELU, Swish, Mish** - smooth, used in modern architectures

No universal winner - **ReLU** and **LeakyReLU** cover most practical needs

Summary

Randomness pervasive in deep learning training - not a nuisance but a feature

Source	Where	Effect
Weight initialisation	start of training	which minimum is found
Data shuffling	each epoch	different batches, prevents order memorisation
Mini-batch SGD	every update step	implicit regularisation, flat minima
Dropout	forward pass	ensemble effect, prevents co-adaptation
Data augmentation	data loading	invariance, population coverage

Embracing randomness rather than eliminating it is part of what makes deep networks generalise well

Generalisation:

- True goal - low loss on population
- Generalisation gap - monitor via validation loss
- Deep NNs generalise surprisingly well
 - flat minima, double descent

Regularisation:

- Weight decay - explicit penalty
- Dropout - random subnetworks
- Early stopping - cut training time
- Data augmentation - most powerful

Training stability:

- Batch norm - stable activation distribs
- Gradient clipping - stop exploding grads
- LeakyReLU / ELU - avoid dying ReLU

In PyTorch:

- `nn.Dropout(p)`
- `nn.BatchNorm1d`
- `weight_decay` in `torch.optim`
- `model.train()` / `model.eval()`